

OS CIE-1

AI Interaction Log

Computer Organisation & Operating Systems

Jayachandra Vandanapu

TY IT | Roll No. 33284

Pune Institute of Computer Technology

I used AI during this assignment mainly to understand Linux commands, plan the locality experiment, debug the dashboard, and check my approach. Whenever an answer depended on the current state of my machine, I tested it using actual output from my WSL2 environment. This log records examples of AI guidance, how I checked it, and what I learned from the results.

1. Entry 1 – AI Prediction Was Incorrect

My prompt

What will happen when I check the process with the highest CPU usage using `ps`? Should I expect the highest CPU value to be greater than 0.0%?

AI answer

The AI predicted that the process with the highest CPU usage would have a CPU value greater than 0.0%.

How I checked

I ran the command directly on my WSL2 system:

```
ps -eo pid,comm,stat,%cpu,%mem --sort=-%cpu | head -n 6
```

Evidence

PID	COMMAND	STAT	%CPU	%MEM
1	systemd	Ss	0.0	0.3
93	systemd-udevd	Ss	0.0	0.1
289	Relay(290)	S	0.0	0.0
51	systemd-journal	S<s	0.0	0.4
196	unattended-upgr	Ssl	0.0	0.5

```
vandanajay-16CHANDR@~/system-detective$ ps -elf
F S UID      PID     PPID    C PRI  NI ADDR SZ WCHAN  STIME TTY          TIME CMD
4 S root        1         0  0  80  0 -   694 -   15:59 ?        00:00:03 /usr/lib/systemd/systemd --system --deserialize=50
4 S root        2         1  0  80  0 -   694 -   15:59 ?        00:00:00 /init
4 S root        6         2  0  80  0 -   694 -   15:59 ?        00:00:00 planD --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
4 S root       158         1  0  80  0 -  1059 -   15:59 ?        00:00:00 /usr/sbin/cron -f -p
4 S message+   159         1  0  80  0 -  2488 -   15:59 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
4 S root       169         1  0  80  0 -   494 -   15:59 ?        00:00:00 /usr/lib/systemd/systemd-logind
4 S root       172         1  0  80  0 - 438960 -   15:59 ?        00:00:00 /usr/libexec/ssl-proc-service -vv
4 S root       178         1  0  80  0 -   798 -   15:59 hvc0      00:00:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
4 S root       182         1  0  80  0 -   779 -   15:59 tty1      00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
4 S root       194         1  0  80  0 - 26756 -   15:59 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
4 S root       320         2  0  80  0 -   695 -   15:59 ?        00:00:00 /init
4 S root       321       320  0  80  0 -   695 -   15:59 ?        00:00:00 /init
4 S vanda      322       321  0  80  0 - 1518 do_wai 15:59 pts/0    00:00:00 -bash
4 S root       323         2  0  80  0 -  1677 -   15:59 pts/1      00:00:00 /bin/login -f
4 S vanda      412         1  0  80  0 -   5811 -   15:59 ?        00:00:00 /usr/lib/systemd/systemd --user --deserialize=8
4 S vanda      413       412  0  80  0 -   5286 -   15:59 ?        00:00:00 (sd-pam)
4 S vanda      428       323  0  80  0 - 1518 core.s 15:59 pts/1      00:00:00 -bash
4 S root      2048         1  0  79 -1 -12585 -   16:03 ?        00:00:00 /usr/lib/systemd/systemd-journald
4 S systemd+  2108         1  0  80  0 - 22756 -   16:03 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
4 S root      2169         1  0  80  0 -   6927 -   16:03 ?        00:00:00 /usr/lib/systemd/systemd-udevd
4 S systemd+  2219         1  0  80  0 -   5364 -   16:03 ?        00:00:00 /usr/lib/systemd/systemd-resolved
4 S polkitd    3356         1  0  80  0 - 77841 -   16:04 ?        00:00:00 /usr/lib/polkit-1/polkitd --no-debug
4 S root      13484         1  0  80  0 - 92521 -   16:05 ?        00:00:00 /usr/libexec/packagekitd
4 S syslog    14111         1  0  80  0 - 55626 -   16:05 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
0 R vanda     18538       322  0  80  0 - 20977 -   16:08 pts/0    00:00:00 ps -elf
```

Figure 1: Process CPU usage observed using the `ps` command.

Conclusion

The prediction was incorrect. The process list was sorted by CPU usage, but every process shown in this snapshot had a CPU usage of 0.0%. Therefore, the highest CPU usage in this particular snapshot was also 0.0%.

This showed me that current process information cannot be assumed from a general prediction and should be checked on the actual system.

2. Entry 2 – Locality Guidance Was Verified

My prompt

How should I perform a row-major versus column-major memory locality experiment in C?

AI answer

The AI explained that C stores two-dimensional arrays in row-major order. It suggested comparing row-major and column-major traversal and measuring their execution times over multiple runs.

How I checked

I implemented the experiment in C using a 4096×4096 array of `double` values and compiled it using:

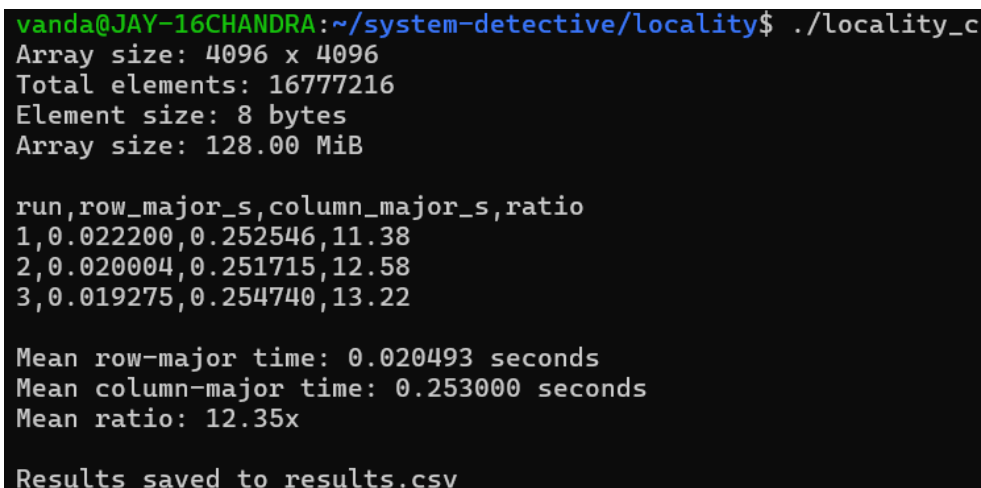
```
gcc -O2 locality.c -o locality_c
```

I then ran both traversal methods three times.

Evidence

The Day 7 checkpoint produced:

```
run,row_major_s,column_major_s,ratio
1,0.022200,0.252546,11.38
2,0.020004,0.251715,12.58
3,0.019275,0.254740,13.22
mean,0.020493,0.253000,12.35
```



A terminal window showing the execution of the `locality_c` program. The output displays array statistics and benchmark results for three runs. The mean row-major time is 0.020493 seconds, the mean column-major time is 0.253000 seconds, and the mean ratio is 12.35x. The results are saved to `results.csv`.

```
vanda@JAY-16CHANDRA:~/system-detective/locality$ ./locality_c
Array size: 4096 x 4096
Total elements: 16777216
Element size: 8 bytes
Array size: 128.00 MiB

run,row_major_s,column_major_s,ratio
1,0.022200,0.252546,11.38
2,0.020004,0.251715,12.58
3,0.019275,0.254740,13.22

Mean row-major time: 0.020493 seconds
Mean column-major time: 0.253000 seconds
Mean ratio: 12.35x

Results saved to results.csv
```

Figure 2: Memory locality benchmark results from the project.

Conclusion

The general explanation was correct. Row-major traversal was faster in all three runs. The exact times changed between runs, but the overall pattern remained the same.

3. Entry 3 – AI Guidance for the Dashboard

My prompt

What is a simple way to create the System Detective dashboard without using a large framework?

AI answer

The AI suggested using a simple Node.js server with HTML, CSS and JavaScript. The server could execute Linux commands and provide the results to the frontend.

How I used it

I implemented the dashboard using the following structure:

```
dashboard/  
|-- server.js  
'-- public/  
    |-- index.html  
    |-- style.css  
    '-- script.js
```

The dashboard displays CPU information, memory usage, the top five processes and locality results. I also added an option to run the locality experiment from the dashboard.

How I checked it

I started the server in WSL2 using:

```
cd ~/system-detective/dashboard  
node server.js
```

I then tested the locality endpoint using:

```
curl -X POST http://localhost:3000/api/locality/run
```

Result

The dashboard successfully displayed the system information and the actual locality benchmark results. The benchmark was executed by the backend rather than using hardcoded values.

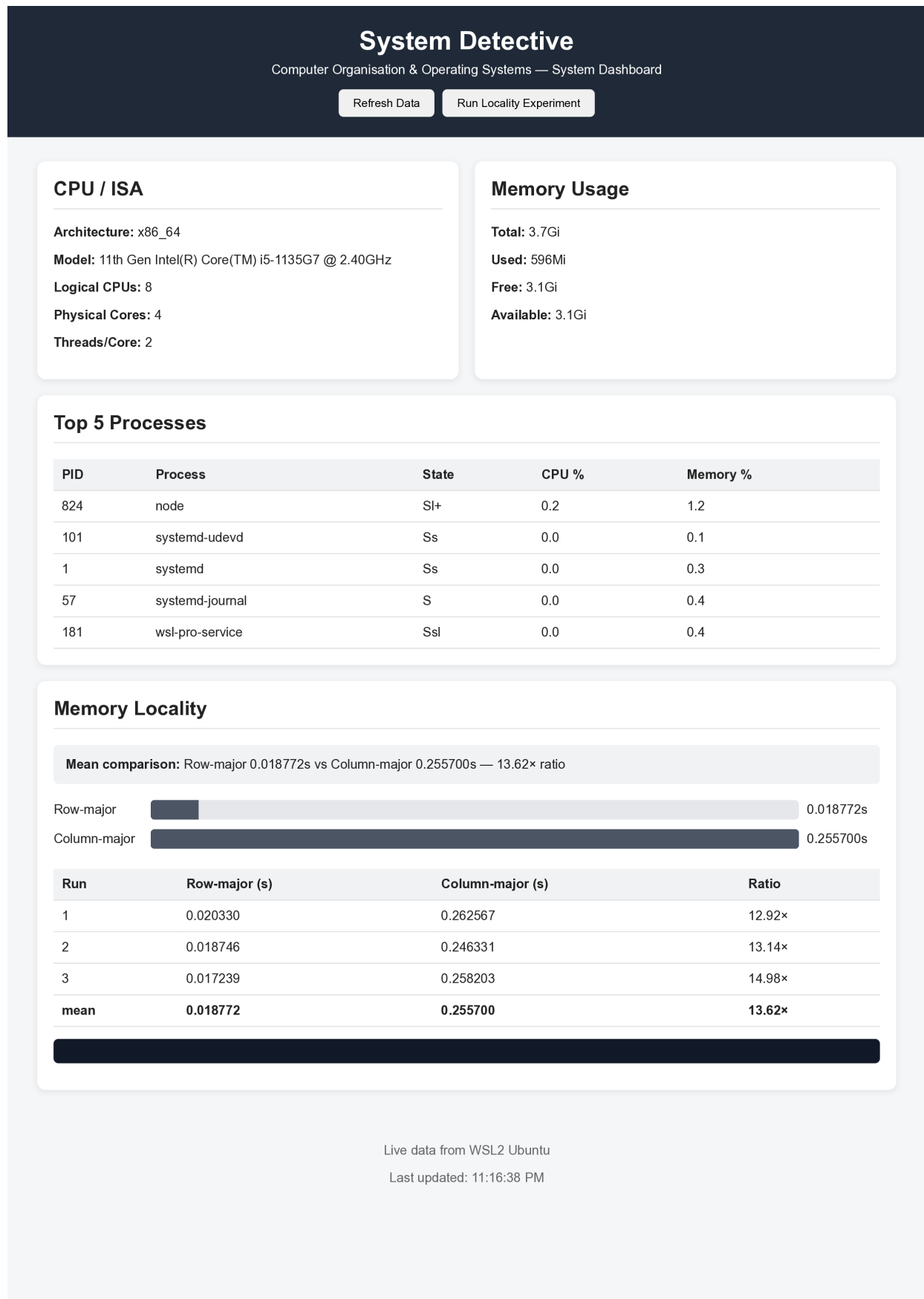


Figure 3: System Detective dashboard developed with AI-assisted guidance.

4. Entry 4 – Reproducibility Guidance Was Verified

My prompt

How can I check whether the locality experiment is reproducible?

AI answer

The AI suggested running the same benchmark multiple times with the same array size, program and compiler settings. It also pointed out that exact execution times can change because the system is running other processes at the same time.

How I checked

I repeated the same benchmark using the same C program, array size, compiler optimization and number of runs.

The latest API execution produced:

```
run,row_major_s,column_major_s,ratio
1,0.020330,0.262567,12.92
2,0.018746,0.246331,13.14
3,0.017239,0.258203,14.98

Mean row-major time: 0.018772 seconds
Mean column-major time: 0.255700 seconds
Mean ratio: 13.62x
```

Conclusion

The exact values were different from the earlier checkpoint, but row-major traversal was still faster than column-major traversal. This confirmed that the timings can vary between executions while the overall locality behaviour remains consistent.

5. What I Learned

The main thing I learned from using AI in this project is that it is useful for explaining concepts and suggesting approaches, but it does not know the current state of my machine.

The incorrect CPU prediction was the clearest example. The AI gave a reasonable-sounding prediction, but the actual command output showed something different.

For this project, my practical rule was simple: use AI to understand what to test, but use my own Linux commands to find out what is actually happening on my machine.

The system-specific numbers in the project were obtained from the actual WSL2 environment rather than being taken from AI responses.